KPlaceNet: understand Kerala mode

Study guide / 21 October 2026

1. What the project does

Visual geolocation means estimating where a photograph was taken from its visible content. KPlaceNet takes one image, analyzes its appearance and returns a location estimate. This guide covers only the Kerala-area system and its regional method choices.

The input is the photograph's pixels. The app does not use the uploaded image's GPS metadata to obtain the answer, read the gallery caption as a label, search the live web for the image or move around a panorama.

A **coordinate** is a latitude-longitude pair. Latitude measures north/south position and longitude measures east/west position. Coordinates such as 9.6761, 76.4211 identify a point on Earth. The number of displayed decimal places does not establish location accuracy.

The three regional choices

The current interface uses a **Regional prediction method** selector. When we say the KNN toggle, we mean choosing the middle option below instead of classifier only. It is not a separate checkbox in this version.

Interface choice	How it selects a location	What it returns
Fine coordinates - ResNet + DINOv2	Find one most-similar training reference using two image encoders	That reference photograph's stored GPS
Regional cells - ResNet + KNN	Blend classifier scores with five training-neighbor class votes	Center of the winning geographic cell
Regional cells - classifier only	Use the classifier's highest-scoring cell	Center of that cell

Classification means choosing a category. A **geographic cell** is a location category, and a **centroid** is its representative center. **Retrieval** means selecting a stored reference by similarity. **KNN**, K-nearest neighbors, uses similar stored examples to inform a decision.

Fine mode also uses nearest-neighbor search, but it is not the five-neighbor class-voting option. It uses one reference and a different combined representation. Turning the class-voting option off does not disable all nearest-neighbor logic in the application.

Regional mode assumes a Kerala-area input. Its collection rectangle includes nearby territory outside the state's boundary. It cannot verify that a foreign or out-of-region photograph belongs to its coverage.

Checkpoint: First distinguish the method, the score and the coordinate source. A retrieved photo coordinate and a cell center are different kinds of output.

2. Data, labels and honest separation

What one data record contains

A **dataset** is a collection of examples. A KPlaceNet record associates an image file with its known latitude and longitude. An image ID identifies the record; it is not a visual feature. A **label** is the answer supplied during supervised learning: here it can be a geographic class derived from GPS.

The main regional imagery comes from **OSV-5M**, a geotagged street-image dataset. **Geotagged** means an image has associated geographic coordinates. The regional selection uses approximately 8.0-12.8 degrees latitude and 74.5-77.5 degrees longitude. This is a collection filter, not a guarantee that every photograph is inside Kerala or that its coordinate is perfect.

Regional split	Number	Purpose
Training	15,939	Learn model parameters and populate retrieval references
Validation	1,000	Choose settings without fitting on the official test
Official test	357	Measure distance error after choosing the method

A **split** is one of these distinct subsets. No validation or test image IDs are in the deployed retrieval bank. However, different IDs can still show almost the same place: a nearby road frame can be easier to match than an unfamiliar location.

Three different kinds of leakage

Data leakage means information that should be held out influences learning or evaluation. Exact-image leakage is reusing the same photo. Spatial leakage is letting very nearby places appear across splits. Selection leakage is repeatedly using test results to decide what to deploy.

In our audit, 982 of 1,000 random validation positions are within 1 km of a training position. None of the 357 official test positions is within 1 km of the original training references. Validation therefore measures an easier setting with many nearby places. Its stronger result must not be presented as the official test result.

The official test has been inspected across multiple project experiments. The current result is a useful measured follow-up, but it is not a newly collected independent confirmation. An independent route/date test would provide stronger evidence.

Checkpoint: Why can two disjoint image-ID lists still produce an optimistic evaluation? Because disjoint files do not guarantee different places or capture sequences.

3. Turning a photograph into model input

Pixels, channels and tensors

A **pixel** is one small color sample in an image. **RGB** represents color using red, green and blue channels. The app converts the uploaded photograph to RGB, including images originally stored in other color formats.

A **tensor** is a multidimensional array of numbers. A single model input has shape 1 x 3 x 224 x 224: one image, three color channels, and 224 rows and columns. **Shape** describes the sizes of an array's dimensions.

The inference preprocessing steps

Preprocessing means applying consistent transformations before a model reads an image.

1. Resize the short edge to 256 pixels while preserving the aspect ratio, so the image is not stretched.

2. Take a central 224 x 224 crop. A crop selects a portion of the image; this can remove useful signs near the edges.

3. Convert pixel values to floating-point numbers, initially in the range 0 to 1.

4. Normalize each color channel using ImageNet mean and standard deviation values.

ImageNet is an image-recognition dataset used in pretraining. The stored channel means are [0.485, 0.456, 0.406], and standard deviations are [0.229, 0.224, 0.225]. For a red-channel value of 0.5, normalization gives (0.5 - 0.485) / 0.229, approximately 0.066. A **standard deviation** describes variation around a mean. This transformation puts input values on the scale expected by the encoders.

Both regional encoders receive this same processed image tensor. The bank was built using matching evaluation preprocessing. If query images and reference images were processed differently, their descriptors could become less comparable.

Training images can be transformed differently

Data augmentation makes altered training views, such as a random crop or horizontal flip, to reduce reliance on one exact appearance. The project's general training transform supports these changes. Inference and cached-feature extraction use the deterministic resize-and-center-crop pipeline instead. **Deterministic** means the same transformation is applied consistently, rather than drawing a fresh random view each time.

Checkpoint: Image normalization changes pixel scales. Descriptor normalization, introduced later, changes feature-vector lengths. They are different operations.

4. ResNet: learning useful visual features

Neural networks and weights

A **neural network** is a sequence of numerical operations with adjustable parameters called **weights**. Training adjusts those weights so useful patterns become easier to recognize. Inference applies the saved weights to an input without changing them.

A **convolutional neural network**, or CNN, learns small filters that are applied across image regions. A **filter** combines nearby pixel or feature values to detect a pattern. Early layers can respond to edges and textures; deeper layers combine patterns into more complex representations. This is learned computation, not a manually coded rule saying that coconut trees imply Kerala.

Why ResNet-50?

ResNet means residual network. A residual connection adds a block's input to a learned transformation: output = input + F(input), when dimensions match. This allows a block to learn a change to its input and helps optimization in a deep network. The 50 identifies the architecture's conventional depth; it does not describe the number of places predicted.

The project uses a ResNet-50 visual backbone initialized from pretrained image-recognition weights and adapted for geolocation. A **backbone** is the main feature extractor. **Pretraining** means learning useful visual representations before our task. **Transfer learning** reuses that knowledge. **Fine-tuning** adjusts some pretrained weights for a new task; the project supports adapting the final ResNet block while earlier layers remain fixed.

The original object-classification layer is removed. After the convolutional blocks, **global average pooling** averages each feature map across its spatial positions. A feature map is a grid of responses produced by learned filters. Pooling produces a 2,048-number vector for each image.

Descriptor versus classifier head

A **descriptor**, also called an embedding or feature vector, is this compact numerical representation of the image. Its 2,048 values are learned features, not 2,048 named landmarks or geographic coordinates.

A **classifier head** is a separate linear layer that maps the descriptor to geographic class scores. Fine retrieval uses the ResNet descriptor itself. The regional cell options use the head as well. The app's shared regional code may compute head scores during a fine request, but those scores do not choose the fine coordinate.

Checkpoint: Why is ResNet useful in retrieval even when its classifier head does not select the answer? Its trained backbone supplies the representation used to compare photographs.

5. DINOv2: a second visual representation

What DINOv2 contributes

DINOv2 is a pretrained visual encoder developed through **self-supervised learning**. Self-supervision builds a learning signal from the images themselves rather than requiring a human geographic label for every training example. The current KPlaceNet uses the official pretrained DINOv2 encoder as a feature extractor; it does not train DINOv2 from scratch or fine-tune its weights on Kerala images.

The installed variant is **ViT-S/14**. ViT means **Vision Transformer**, S denotes the small variant, and 14 is the patch width in pixels for this model. At our 224 x 224 input size, dividing the image into 14 x 14 patches creates a 16 x 16 arrangement of 256 patches.

A **patch** is a small image region. A **token** is a numerical representation of a patch or special model element. A transformer processes relationships among tokens. **Attention** lets a token combine information from other tokens with learned weights; it is not a human explanation or proof that the model noticed the correct landmark.

The model produces a 384-number image descriptor. The project uses the complete image descriptor rather than matching individual patch coordinates or reconstructing a 3D scene.

Why use two encoders?

ResNet and DINOv2 encode appearance differently. Combining them lets both representations contribute to reference matching. This can help when one encoder's strongest similarity is misleading. It can also fail if both encoders find the same wrong-looking place.

In deployment both encoders are **frozen**, meaning their weights remain fixed. Offline reference extraction saves their descriptors. For an uploaded query, the app computes a new descriptor with each encoder. There is no parameter update when you click Locate photo.

Ensemble means combining multiple model outputs or representations. Our fine method is a descriptor ensemble: it combines normalized features before searching. It is not a vote between two independently predicted latitude-longitude pairs.

Checkpoint: DINOv2 does not supply a second GPS prediction. It supplies another visual descriptor. The bank supplies the returned GPS coordinate.

Source for DINOv2's model family and pretrained encoders: official DINOv2 repository.

6. Combining features: a worked explanation

Why normalize a descriptor?

A vector's **L2 norm** is its length: the square root of the sum of its squared values. L2 normalization divides every value by this length, producing a unit-length vector. For example, [3, 4] has length 5, so its normalized version is [0.6, 0.8].

This keeps vector magnitude from dominating a comparison. It also prevents the 2,048-dimensional ResNet descriptor from automatically outweighing the 384-dimensional DINO descriptor merely because it has more components.

Let r be the normalized ResNet descriptor and d be the normalized DINO descriptor. **Concatenation** means placing one vector after the other to create a longer vector. Our combined descriptor is:

`z = [sqrt(0.5) r, sqrt(0.5) d]`

Its dimension is 2,048 + 384 = 2,432. Each part has half of the squared length, so the combined vector has unit length before storage rounding. The app normalizes again when searching.

What cosine similarity measures

Cosine similarity measures how closely two vectors point in the same direction. It is their dot product divided by their lengths. A **dot product** multiplies corresponding components and adds the products. For unit vectors, the dot product is already cosine similarity.

Because of the weighting above, the combined similarity is equally influenced by the ResNet and DINO similarities:

`combined similarity = 0.5 ResNet cosine + 0.5 DINO cosine`

Illustrative example, not actual project measurements:

Reference	ResNet similarity	DINO similarity	Combined
A	0.90	0.50	0.70
B	0.75	0.80	0.775

ResNet alone prefers A. The combined descriptor prefers B. This demonstrates how the second encoder can change the match without any geographic coordinate averaging.

A cosine score such as 0.812 is a similarity value. It does not mean an 81.2% chance that the location is correct, nor does it specify a distance error. The project's fine pipeline has no calibrated conversion from similarity to probability.

Checkpoint: Explain why the square-root weights result in equal similarity contributions: multiplying query and reference weights gives sqrt(0.5) x sqrt(0.5) = 0.5.

7. Fine-coordinate retrieval, end to end

Query and reference bank

A **query** is the image whose location we want to estimate. A **reference** is a training image with a known coordinate. A **reference bank** stores each reference's descriptor, image ID and coordinate. The deployed fine bank contains 15,939 original training entries. Some may have the same coordinates; this is not a claim of 15,939 distinct places.

The query and references pass through matching encoders and preprocessing. Their descriptors can therefore be compared in the same **feature space**, meaning a shared numerical representation where similarity is computed.

For every reference j, the app calculates a cosine score s(j). It selects the largest score using **argmax**, which means the index of the largest value. The selected index retrieves the saved latitude and longitude. This is **top-one nearest-neighbor retrieval**: one best reference supplies the answer.

What is actually predicted?

The neural networks predict descriptors; the search selects a reference; the lookup returns that reference's GPS. The result is not restricted to 300 class centers. It is also not **coordinate regression**, which would directly produce numerical latitude and longitude through a learned output function.

Fine mode does not average the top five reference coordinates, use classifier probabilities to adjust the chosen fine coordinate or run geometric verification. **Geometric verification** would test whether detailed image correspondences support a physically consistent scene match. That is a possible extension, not an installed feature.

Why retrieval still has a precision limit

If the bank has no relevant nearby photograph, even the best match can be distant. A top-one output cannot be closer to the true location than the closest bank coordinate, and appearance matching can select something much farther away. Increasing coverage with accurate, varied references is therefore as important as improving the encoder.

This is not a direct pixel comparison: the query can differ in lighting or viewpoint and still have a similar learned descriptor. Conversely, two different places can share a similar descriptor and produce an incorrect match.

Checkpoint: The returned coordinate belongs to the reference photo. Without the query's true coordinate, we cannot know the query-reference distance error just by reading the output.

8. How classifier-only prediction works

Geographic cells are training labels

The regional classifier has 300 **classes**, or output categories. Each class corresponds to a group of training GPS positions with a stored center and latitude-longitude bounds. They are not districts, recognized town names or uniform-size squares.

The groups are created with **K-means**, a clustering algorithm. A cluster is a group of examples. K-means assigns positions to nearby cluster centers and updates those centers repeatedly. In this project it groups geographic coordinates, not image colors or image descriptors. The resulting cell metadata is saved and reused during classification.

K-means and KNN are different. K-means builds geographic groups offline. KNN searches visual descriptors of known training images during a prediction. The 300 classes and the five neighbors count different things.

From a ResNet descriptor to class scores

The ResNet backbone supplies 2,048 visual features. A **linear classifier head** applies learned weights and biases to these features, producing one score per cell. In compact form, scores = W x + b, where x is the descriptor, W contains learned feature weights and b contains learned offsets.

These raw scores are called **logits**. They are unrestricted real numbers. **Softmax** converts them into nonnegative class scores summing to one:

```
p(j) = exp(logit(j)) / sum(exp(all logits))
```

Here exp is the exponential function, and j identifies a class. **Argmax** selects the index with the largest score. If cell 196 wins, the app returns the saved center of cell 196. It does not calculate a new GPS pair from the image.

What the score and box mean

The largest softmax score indicates how strongly the classifier favors one class relative to its other classes. A high score is not automatically a well-calibrated probability of being correct. **Calibration** means checking whether scores agree with observed correctness over suitable labeled examples. Regional scores are not calibrated.

A cell's displayed bounding rectangle is a geographic extent derived from its associated positions. It is not a confidence interval, and it does not guarantee that the query is inside. A confidence interval would need a separately justified uncertainty procedure.

All photos assigned the same winning cell receive that same center coordinate. This is why the cell options have a geographic resolution limit even when they choose the right general area.

Checkpoint: The classifier learns a decision rule from image features. At inference it chooses a cell using learned weights; it does not need to look up a particular training photograph.

9. What enabling KNN actually does

Start with the same ResNet features

Select **Regional cells - ResNet + KNN** to use five-neighbor class voting. The classifier still runs. KNN adds evidence from stored training examples; it does not replace ResNet or run a second neural classifier.

A **query** is the uploaded image. A **reference** is a training image with a known cell label. A **bank** is the saved collection of reference descriptors and labels. The class-voting bank contains the 15,939 training entries. No validation or test image IDs are included.

The complete five-step procedure

1. Extract the query's 2,048-dimensional ResNet descriptor. A descriptor is a learned numerical description of appearance.

2. L2-normalize query and reference descriptors to unit length, then compute cosine similarities. This compares vector direction rather than raw length.

3. Select the five references with highest similarity. The number k = 5 counts reference photographs, not classes. Several references can share the same class label.

4. Convert the five similarities into normalized weights and add their weights by geographic class. The implementation uses softmax(similarity / 0.07). The constant 0.07 controls how strongly the higher-similarity neighbors dominate. It is a score-sharpening setting, not fitted confidence calibration.

5. Blend the neighbor votes with classifier scores, choose the largest blended score and return that class's center.

The blend for each cell j is:

$$combined(j) = 0.8\ classifier(j) + 0.2\ neighbor_vote(j)$$

The classifier therefore contributes 80% of the score distribution and the neighbors 20%. This is not an 80% accuracy claim. A **vote distribution** means the class totals formed from the five neighbor weights; those totals sum to one.

What changes when KNN is disabled?

Select **Regional cells - classifier only**. The app selects the class from classifier scores alone. The ResNet input and learned classifier are the same; the neighbor vote no longer influences the decision. Both options still return one of 300 stored centers.

Neither option changes model weights when a photo is uploaded. KNN uses stored examples at inference rather than fitting a new neural network on each query. It cannot tell whether a visually similar reference is geographically correct.

Checkpoint: KNN changes the evidence used to choose a cell. It does not refine a cell center into a more precise point within that cell.

10. KNN: a numerical example

The numbers here are illustrative teaching values, not outputs from a particular project photograph.

Neighbor weights become class votes

Suppose the five most similar references have these already-normalized weights. The weight depends on visual similarity, not on whether the place is truly correct.

Neighbor	Stored cell	Weight
1	B	0.30
2	B	0.25
3	A	0.20
4	B	0.15
5	C	0.10

The weights sum to one. Add weights belonging to the same cell: A receives 0.20; B receives 0.30 + 0.25 + 0.15 = 0.70; C receives 0.10. Five neighbors have produced votes for three classes.

Blend these votes with classifier scores

For simplicity this example uses three classes. The real model has 300.

Cell	Classifier score	Neighbor vote	Combined score
A	0.48	0.20	0.8 x 0.48 + 0.2 x 0.20 = 0.424
B	0.45	0.70	0.8 x 0.45 + 0.2 x 0.70 = 0.500
C	0.07	0.10	0.8 x 0.07 + 0.2 x 0.10 = 0.076

KNN disabled: A wins with classifier score 0.48. The coordinate is A's center.

KNN enabled: B wins with combined score 0.500. The coordinate becomes B's center. The result did not move toward a weighted average of the five reference GPS positions; a different cell won.

Why KNN does not always change the answer

If the classifier strongly prefers A, a 20% neighbor contribution may be too small to reverse the ranking. If both sources already favor A, the cell can stay unchanged while its score changes. An unchanged marker does not mean the toggle is broken.

KNN can also make the answer worse: several visually similar but distant references may vote for the wrong cell. Similar appearance is useful evidence, not proof of geographic identity. The displayed blended score is still uncalibrated.

Checkpoint: Explain what is being averaged: class-score distributions, not the coordinates themselves and not the neural-network weights.

11. How the learned model is trained

Supervised learning and loss

Supervised learning uses examples with known answers. For geographic classification, a training photo's GPS position determines its target cell label. The network receives image pixels; GPS is used to define the answer during training, not to give away a query's location at inference.

A **forward pass** computes the network's scores. A **loss** measures disagreement with the target. **Cross-entropy** penalizes placing little probability on the correct class. With one hard target y, the loss is -log p(y). Assigning p(y) = 0.8 gives a lower loss than p(y) = 0.1.

Backpropagation computes how each trainable weight affects the loss. A **gradient** is that rate of change. An **optimizer** uses gradients to update weights. The **learning rate** controls the update step size. A **batch** is a group processed together; an **epoch** is one pass through the chosen training examples.

The currently installed regional head

The deployed ResNet descriptor comes from the geolocation-adapted backbone. Its installed classification head was fitted while keeping that backbone fixed. **Freezing** weights means not optimizing them. This lets the project cache the 2,048 features and train the head cheaply.

Its **geographic label smoothing** uses a soft target: 90% assigned class plus 10% distributed according to distance from the training GPS to cell centers. Nearby centers get more of that remaining mass, using a 15-km distance scale. A **soft target** distributes credit across classes rather than assigning all credit to one. This can reduce an overly sharp penalty at arbitrary geographic boundaries.

AdamW is an optimizer that adapts updates using running gradient statistics and applies weight decay separately. The head-fitting configuration is AdamW, learning rate 0.0001, weight decay 0.01, batch size 256 and 12 epochs per recipe. **Weight decay** discourages unnecessarily large weights; it is a form of regularization, meaning a constraint intended to improve generalization. These settings describe head fitting, not a fresh end-to-end training of both fine-retrieval encoders.

The fine coordinate is selected from frozen descriptors, so changing the head alone does not directly change the deployed fine search.

Checkpoint: Training changes parameters. Clicking Locate only runs inference and lookup; it does not teach the model a new place.

12. Building and selecting the retrieval system

Offline work versus a live prediction

Offline here means preparation performed before serving a user query, not necessarily without internet. Build the bank by applying the frozen ResNet and DINO encoders to each training reference once. Save the combined descriptors, coordinates and IDs. A **cache** is saved work reused later instead of recomputing it.

At inference, only the query needs fresh encoder passes. Comparing one 2,432-component descriptor against 15,939 cached references is a matrix multiplication. A matrix is a table of numbers; multiplying the query by the reference matrix calculates all similarity scores efficiently.

The bank stores descriptors in **FP16**, a 16-bit floating-point format that saves space. At search time the app converts them to FP32, a higher-precision 32-bit format, and normalizes them. Storing fewer bits can introduce small rounding differences, so the compressed deployment bank was checked separately. Its official-test threshold counts and median match the reported current result.

What was selected, and how?

A **hyperparameter** is a setting chosen rather than learned by gradient updates: for example, encoder weight or neighbor count. The current equal encoder weighting and k = 1 were selected from six fixed mixture/neighborhood options on validation.

The main selection objective was the percentage within 5 km, with a median-error tie-break and a guardrail against excessive worsening within 25 km. A **guardrail** is a minimum acceptable condition on another metric. The winning method was fixed before its test evaluation. The reused-test caveat still applies because the overall project has examined that benchmark repeatedly.

Why not store only training images and search them later?

Neural feature extraction is the expensive part. Recomputing every reference descriptor per query would waste time. The bank makes inference practical. Keeping query and reference encoder versions consistent is essential: a descriptor bank from a different encoder may no longer be comparable.

A **checkpoint** is a saved model state. A **SHA-256 fingerprint** is a file-content digest used to check identity. Installers verify asset fingerprints, and runtime checks tie the fine bank to the matching ResNet and DINO weights. This prevents accidental resource mismatch; it does not prove predictive accuracy.

Checkpoint: What needs rebuilding after an encoder's weights or preprocessing changes? The reference descriptors, and the validation/evaluation needed to justify the new system.

13. Reading the actual regional result

Distance error and thresholds

Ground truth is the recorded correct coordinate used for evaluation. Distance error compares the predicted coordinate with that recorded position. The project uses **haversine distance**, a great-circle distance calculation on a spherical Earth approximation. A great-circle path is the shortest surface path on a sphere; simply subtracting latitude and longitude degrees would not give kilometers.

For each labeled query, calculate one error in kilometers. Then summarize the distribution. A **metric** is a numerical performance measure.

Current fine mode: 357 official test photos	Result
Within 1 km	0 of 357
Within 5 km	62 of 357: 17.37%
Within 25 km	225 of 357: 63.03%
Within 200 km	90.76%
Median error	15.92 km
Mean error	61.21 km
90th-percentile error	197.75 km

Median is the middle of the ordered errors; roughly half fall on each side. **Mean** is the arithmetic average and is pulled upward by large failures. The **90th percentile** is an error cutoff at roughly the 90% position of the ordered results; it is not the accuracy of one uploaded photograph.

"63.03% within 25 km" means 225 queries had an error at most 25 km. It does not mean 63.03% were on the exact street. Reporting an accuracy without its distance threshold is misleading for this task.

Why zero within 1 km?

Every official test position is more than 1 km from the original reference bank, and fine mode returns a reference coordinate. That creates a geometric ceiling: no possible top-one match from this bank can meet the 1-km threshold on this subset. The reference coverage must change, or the output method must estimate beyond reference coordinates, to overcome that ceiling.

Random validation gives 71.1% within 5 km and 1.07-km median, but contains many near-training positions. Those numbers are not interchangeable with the official test. These figures apply to regional fine retrieval, not the cell options.

Checkpoint: Why can the median be 15.92 km while the mean is 61.21 km? A smaller number of very distant errors can strongly increase the mean.

14. Failure, generalization and overfitting

Four different reasons for a wrong location

Coverage failure: no geographically useful reference exists. **Appearance ambiguity:** the bank has relevant places, but another location looks more similar. **Domain shift:** the query differs from training imagery, such as a drone view, indoor photo, heavy fog or a famous foreign landmark. **Label error:** the recorded reference or test GPS is inaccurate.

A **domain** is a data setting, such as street-level camera imagery in a region. **Out-of-distribution**, or OOD, describes inputs outside the patterns the model was developed for. The regional system has no validated OOD rejection method. Giving it the Great Wall of China can still return an Indian reference because the algorithm always selects its highest available similarity.

What overfitting means

Overfitting occurs when learning or method selection exploits details of development data that do not generalize to new data. **Generalization** means performing well on genuinely new cases. A curated success cannot prove generalization, and a single foreign-image failure cannot by itself establish training overfitting.

The retrieval bank intentionally stores training references. Storing them is how retrieval works, not automatically evidence of improper memorization. The important questions are whether held-out queries were excluded, whether scenes or sequences overlap across splits, and how performance changes on new routes and conditions.

Current warnings are specific: random validation has many nearby references; official test is spatially harder; and the benchmark was reused. These limit the strength of the performance claim even though the bank has no held-out IDs.

Explainability versus verification

Showing the retrieved photograph provides **inspectability**: a person can compare the visual match and see where the coordinate came from. It is not a complete explanation of all neural features and is not geometric proof of place identity. Attention is not a causal explanation either.

A local map makes the result visible; it cannot make it correct. The Google Maps link can help inspect roads and surroundings but does not retrospectively certify the model output. A displayed similarity should therefore always be described as similarity.

Checkpoint: Diagnose a northern match for a southern road photo as an observed retrieval error. Do not invent a reason such as "the model read this sign" without testing that claim.

15. Reading the Kerala frontend correctly

The controls and their effects

The **frontend** is the user-facing interface; the **backend** is the Python prediction logic. Gradio connects controls to Python callbacks. A callback is a function triggered by an action. Plotly draws the interactive map.

Keep geographic scope on **Kerala area - regional model**. Use **Regional prediction method** for the three regional choices. To demonstrate the KNN toggle, compare **Regional cells - classifier only** against **Regional cells - ResNet + KNN**, using the same loaded photograph and clicking Locate after each change.

Fine coordinates is a third, separate choice. It uses two encoders and top-one reference GPS lookup. Do not describe switching from classifier only to fine mode as merely enabling the five-neighbor KNN blend.

An upload supplies image pixels. A gallery click loads a selected image and its scope but does not itself run inference. **Locate photo** performs prediction. Changing an image or method clears the old result, so a stale answer is not shown as the new method's output.

Score labels and coordinates

In fine mode, the output says **Retrieved location** and **Appearance similarity (cosine)**. The separate reference picture is a training photograph, not the uploaded query. Its saved GPS supplies the marker position. A score such as 0.812 is not an 81.2% probability of correctness.

In the cell options, the marker is a cell center. The class-plus-KNN details distinguish classifier scores from the selected blended score. Both are uncalibrated. A cell rectangle, if drawn, is a geographic extent and not a confidence interval.

The 300 small background map dots are class centers, including when fine mode is selected. They do not show all 15,939 reference entries. Fine retrieval can return a point that is not one of those background dots.

Examples, Maps and known error

The gallery queries came from official test archives and were deliberately chosen. **Curated** means selected for demonstration, not sampled to estimate average performance. Captions help the presenter; they are not sent to the image encoder as the answer.

Open in Google Maps displays the predicted coordinate outside the app. It does not search for the true place or correct the prediction. Without a known query coordinate, the app cannot measure a new upload's distance error.

The Southern Kerala example is a known fine-mode failure: our checked run matches a northern reference near 12.45793, 75.09220. Show it as an error, not as evidence that any location within the regional map is acceptable.

Checkpoint: For every result, identify which regional method ran, whether the coordinate is reference GPS or a center, and which kind of score is displayed.

16. Running and understanding the implementation

Start this laptop's app

From the task workspace, run:

```
cd ~/Documents/Codex/2026-10-20
cd https-github-com-asterisk-hunter-kplacenet
./outputs/Start-KPlaceNet.sh
```

Open http://127.0.0.1:7861/ and keep the terminal open. **Localhost**, including 127.0.0.1, refers to this machine; 7861 is the service's **port**, a numbered network endpoint. A different device's localhost is a different machine.

From inside the repository at `work/KPlaceNet`, the same launcher is `../../outputs/Start-KPlaceNet.sh`. The launcher uses the installed GPU environment when it works, otherwise the CPU environment. A **GPU** accelerates parallel numerical work. **CUDA** is NVIDIA's GPU-computing platform. Inference can run on CPU more slowly.

A **virtual environment** keeps a project's Python packages separate. The app uses PyTorch for tensor operations and neural networks, Torchvision for ResNet and image transforms, Pillow for image decoding, Gradio for the UI and Plotly for maps. The first prediction is slower because resources are loaded **lazily**, when first needed, then cached in memory. Models do not need to reload for every query.

Files to recognize

File or location	Responsibility
demo/app.py	UI, mode dispatch, model loading, maps and output labels
src/dataset.py	Image loading and transforms
src/model.py	ResNet backbone and class head
src/fine_inference.py	DINO loading, descriptor fusion and fine search
src/retrieval.py	Five-neighbor regional class voting
src/cells.py	Cell construction, assignment and metadata
src/demo_config.py	Regional artifact selection and identity checks

Model weights and banks are **artifacts**, meaning saved resources required at runtime. They are downloaded from releases rather than stored as large Git files. In a separately prepared Python environment, `python scripts/download_fine_retrieval.py` installs the verified fine assets and matching regional model artifacts. It does not install Python packages or download the entire image dataset. Reference thumbnails require their corresponding image files.

Checkpoint: Cloning source code alone is not enough to reproduce the installed demo: packages, weights, cell metadata and matching banks are also needed.

17. Explain Kerala mode in your own words

The explanation to understand, not just memorize

KPlaceNet estimates a photo's location from visual appearance. The app preprocesses the photo into a normalized 224 x 224 RGB input. ResNet transforms it into 2,048 learned features.

In classifier-only mode, a learned head selects one of 300 geographic cells and returns its center. Enabling the regional KNN option finds five visually similar training references, adds their weights by class and blends 20% neighbor votes with 80% classifier scores. The winner still supplies a cell center.

Fine mode follows another path: it combines the normalized ResNet descriptor with a 384-dimensional DINOv2 descriptor, gives both equal similarity weight and retrieves one best training reference. Its coordinate is that reference's saved GPS. It is not an arbitrary new latitude-longitude prediction.

Neither path verifies that the input belongs to Kerala. The collection rectangle includes neighboring territory; visually similar scenes can produce distant wrong answers. Scores are not calibrated correctness probabilities. Test results describe a labeled dataset, not the accuracy of whichever photograph is currently uploaded.

Questions and answers

Why can five neighbors produce fewer than five class votes? Several references may share a cell, so their weights are added together.

Does KNN move the marker inside a cell? No. It can change the selected cell; the output remains that cell's saved center.

Does disabling KNN retrain ResNet? No. It removes the vote blend from that inference path. The learned weights remain fixed.

Is fine mode simply KNN enabled? No. It combines two encoders and returns one reference GPS; the regional class-voting option uses five ResNet neighbors and returns a center.

Why might the toggle leave the result unchanged? Both sources can favor the same cell, or the classifier's lead may be too strong for 20% neighbor votes to overturn.

Is the reference photo allowed to be a training image? Yes. The bank intentionally contains training references. A held-out query must be excluded; nearby scene overlap still needs evaluation controls.

What does 63.03% within 25 km mean? 225 of 357 official test photos have distance error at most 25 km in fine mode. It is not exact-street accuracy and does not describe the cell-plus-KNN method.

Where the explanation comes from

Frontend and method dispatch, five-neighbor class voting, fine retrieval, image preprocessing, ResNet model, and current fine evaluation.